CareerTailor

Full-Stack Development Plan · V1.1 Revised · February 2026

Goal	Ship CareerTailor as a free public product — custom resumes, job search, and one-click apply — on a zero-cost infrastructure stack.
New Features	(1) Custom resume generation from job descriptions (2) One-click job application (3) Free job search via Adzuna API — no LinkedIn required
LinkedIn API?	Do NOT pursue it. Enterprise-only, $3K+/month, months of approval. Answered fully in Section 2.

Table of Contents

01 LinkedIn API — The Real Answer

Should you pay for it? Short answer: No. Here's why.

Bottom Line: LinkedIn's Job API is enterprise-only, costs $3,000+/month, requires a formal LinkedIn partner approval process that takes months, and LinkedIn is currently NOT accepting new partnerships for its Job Posting API. This is a dead end for an indie developer launching a free public product.

What the LinkedIn API Actually Costs

API Type	Access	Estimated Cost	Verdict
Sign In with LinkedIn	Free, self-serve	$0	■ Use this for auth
Basic Profile API	Free, limited fields	$0	■ Usable
Jobs Search API	Partner program only	$3,000–$10,000+/mo	■ Not viable
Job Posting API	Partner only — CLOSED to new applicants	Enterprise contract	■ Completely blocked
Recruiter API	LinkedIn-certified partners only	Tens of thousands/mo	■ Enterprise only
Apply Connect	Must apply separately, approval required	Unknown / negotiated	■ Months of process

What This Means for CareerTailor

Even if you could afford it, LinkedIn's Job API only lets you **post jobs on behalf of employers** — it is not a job search API for candidates. It's built for ATS (Applicant Tracking Systems) and recruiters, not for job seekers. It would not even solve your use case.

For one-click apply specifically: LinkedIn's **Apply Connect** allows users to apply to jobs using their LinkedIn profile — but again this requires approved partner status. The practical alternative is to use the job's direct application URL (which all job APIs return) and open it for the user with pre-filled form data where possible.

02 Free Job API Alternatives

Replace LinkedIn scraping with legitimate, stable, free APIs.

Recommended Primary: Adzuna API — truly free tier, no partner approval, covers 50+ countries, returns full job descriptions, salary data, and direct application URLs. Pair with Greenhouse and Lever public endpoints for tech company jobs specifically.

Tier 1 — Use These First (Truly Free)

API	Free Tier	Coverage	Key Advantage	Apply Support
Adzuna API	Free, self-serve signup	50+ countries, millions of listings	Best free job aggregator. Returns salary, full description, direct URL.	Direct URL returned
Greenhouse Job Board API	100% free — public JSON	Tech companies (Airbnb, Stripe, etc.)	No auth needed. Companies expose their jobs publicly.	Direct application URL
Lever Job Board API	100% free — public JSON	Tech startups & scale-ups	No auth needed. Same pattern as Greenhouse.	Direct application URL
Workday (public JSON)	Free — fetch company-specific	Enterprise companies (Apple, etc.)	Most enterprises use Workday. Parseable JSON endpoints.	Deep link to apply page
USAJOBS API	Free, API key only	US Federal government jobs	Official US govt API. Very stable.	Gov application portal
Remotive API	Free, no auth	Remote jobs globally	Focused on remote/tech roles.	Direct URL returned

Tier 2 — Use as Fallback (Free tiers exist)

API	Free Tier	Notes
Jooble API	Free with API key	Aggregator, 70+ countries. Apply for key at jooble.org/api
The Muse API	Free	Strong for company culture data. Good for mid-market tech jobs.
Reed.co.uk API	Free with registration	Best for UK jobs. Solid coverage.

Arbeitnow API	Free, no auth needed	Strong for European/remote roles. Public JSON.

One-Click Apply — How to Do It Without LinkedIn

Every job API above returns a direct application URL. Your 'Apply' button should:

1. Open the job's direct application URL in a new tab
2. Simultaneously generate & download the tailored resume for that specific job
3. For supported ATS platforms (Greenhouse, Lever), you can use browser autofill-compatible form prefilling — parse their JSON schema and pre-populate name, email, LinkedIn URL from the user's profile stored in Supabase
4. Future enhancement: integrate with browser extension to auto-fill forms (out of scope for V1 but very doable)

Reality check on 'one-click apply': True one-click apply (submitting the application programmatically) is only possible for platforms that expose a candidate-facing apply API — which almost none do publicly. What you CAN do reliably: open the application URL + auto-download the tailored resume in one click, so the user just pastes/uploads it. This is still a massive UX win and honest about what it does.

03 New Features: Custom Resume + One-Click Apply

How these features fit into the existing architecture.

Feature 1 — Custom Resume from Job Description

This is an **extension of the existing Phase C (Generation)**. The difference: instead of just injecting keywords, the AI rewrites or generates entire bullet points using the job description as a creative brief and the user's semantic memory (pgvector) as the source of truth.

FEATURE 1A — **Job Description Input → Resume Rewrite** — New LangGraph node

User Flow:

1. User pastes or provides a job URL (new jobs from Adzuna API, or manual paste)
2. System extracts the full job description (existing scraper node reused)
3. **NEW — Rewrite Node:** LLM receives: (a) job description, (b) RAG-retrieved relevant resume chunks from pgvector, (c) rewrite prompt with tone/style instructions
4. LLM rewrites the relevant bullet points to match the job's language and requirements
5. User previews the changes in a diff-style UI before accepting
6. Accepted version generates a new tailored .docx file

New UI Elements Needed:

Component	Description	Complexity
Resume Diff Viewer	Side-by-side old vs new bullet points. User accepts/rejects each change.	Medium
Job Description Input	Text area + URL paste. Reuses existing scraper.	Low
Rewrite Strength Slider	Conservative (keyword inject only) ↔ Aggressive (full rewrite). Maps to prompt temperature.	Low
Section Toggle	User picks which resume sections to rewrite vs preserve.	Low

FEATURE 1B — **Resume Score vs Job Description** — Bonus feature

Before and after the rewrite, show a match score (0–100%) computed by the LLM comparing the resume against the job description. This is a great engagement hook and gives users a clear before/after sense of

improvement. Reuses the existing Feasibility Score node.

Feature 2 — One-Click Apply

FEATURE 2

One-Click Apply Flow

V1: open + download. V2: autofill.

Version	What Happens on Click	Technical Approach	Effort
V1 (Launch)	Tailored resume downloads + job application URL opens in new tab simultaneously	JS: window.open(applyUrl) + trigger file download. Single button.	1 day
V1.5	User profile data (name, email, phone, LinkedIn) pre-copied to clipboard for fast paste	Supabase user profile table. Clipboard API.	2 days
V2 (Post-launch)	For Greenhouse/Lever jobs: auto-fill application form fields using stored profile	Detect ATS from URL pattern. POST to their JSON apply endpoint where exposed.	1–2 weeks

04 Frontend Development Plan

React + Vite. Deployed on Vercel. Component-by-component breakdown.

Technology Choices

Technology	Choice	Why
Framework	React 18 + Vite	Fastest dev experience. HMR, tree-shaking, optimal bundle size.
Styling	Tailwind CSS	Rapid UI, no CSS files to maintain, consistent design system.
Component Library	shadcn/ui	Unstyled, accessible components. Customize to match dark theme.
State Management	Zustand	Lightweight. Perfect for job queue state, resume state, user session.
Data Fetching	TanStack Query (React Query)	Caching, background refetch, optimistic updates for job status.
Realtime	Supabase JS client	Subscribe to job_status table. No separate WebSocket setup.
Forms	React Hook Form + Zod	Fast, type-safe, minimal re-renders for URL submission forms.
File Handling	react-dropzone	Drag-and-drop resume upload. Handles file type validation.
Diff Viewer	react-diff-viewer-continued	Side-by-side resume diff for the rewrite feature.
Charts	Recharts	Feasibility score chart, match score visualisation. Free, React-native.
Hosting	Vercel (free tier)	Auto-deploys from GitHub. Edge CDN. Custom domain support.

Page & Component Architecture

Page / Route	Key Components	Data Source

/login	Supabase Auth UI, Google OAuth button, Email magic link	Supabase Auth
/dashboard	Recent sessions list, Quick stats (resumes generated, jobs applied), CTA cards	Supabase: sessions table
/scan (new session)	URL input (3–5 fields), Submit button, Job queue status cards with traffic lights	FastAPI + Supabase Realtime
/interview	Chat-style Q&A; interface, Progress indicator, Skip button for auto-verified skills	FastAPI streaming
/results/:sessionId	Job cards with match score, Rewrite button per job, Apply button, Resume preview	Supabase: jobs table
/resume/rewrite	Job description input, Diff viewer (old vs new), Section toggles, Strength slider	FastAPI + Gemini
/jobs	Job search bar, Adzuna API results, Filter panel (location, salary, remote), Save job button	FastAPI → Adzuna API
/profile	User info form, LinkedIn URL, Resume upload history, Danger zone (delete account)	Supabase: users table

Frontend State Architecture

Zustand Store	State Held	Persisted?
authStore	user object, session token, isLoading	Supabase session (auto)
jobQueueStore	jobs[], sessionId, pollingStatus, trafficLights	No — session memory
resumeStore	masterResumeMetadata, selectedJobId, rewriteDiff, pendingChanges	No — session memory
searchStore	searchQuery, filters, adzunaResults, savedJobs[]	savedJobs in Supabase
uiStore	sidebarOpen, activeTab, toasts[], loadingStates	No

05 Backend Development Plan

FastAPI + LangGraph. Deployed on Render. Full endpoint breakdown.

Technology Choices

Technology	Choice	Why
Framework	FastAPI (Python 3.11+)	Async-native, auto OpenAPI docs, Pydantic validation, fast.
AI Orchestration	LangGraph 0.2+	Stateful agent graphs. Retry, branching, parallel nodes built-in.
LLM	Gemini 1.5 Pro (generation)	Best quality/cost for resume rewriting tasks.
Embeddings	Gemini text-embedding-004	768-d. Same provider = simpler billing.
Vector Search	pgvector on Supabase	Free. Already in stack. Cosine similarity, RLS-aware.
Database ORM	SQLAlchemy 2.0 (async) + asyncpg	Async Postgres. Type-safe queries.
Job Queue	FastAPI BackgroundTasks + Supabase	No Redis/Celery needed for V1. Simple and free.
Web Scraping	httpx (primary) + Playwright (fallback)	httpx for lightweight fetches. Playwright only for JS-heavy pages.
Document Processing	python-docx (run-level)	Resume injection at run level preserves all formatting.
Auth	Supabase Auth (JWT verification)	Validate Supabase JWT in FastAPI middleware. One function.
Rate Limiting	slowapi	Per-user limits. In-memory for V1, Redis-backed for V2.
Job APIs	Adzuna (primary) + Greenhouse/Lever JSON	Free. No scraping. Stable.
Hosting	Render (free tier)	Sleeps after 15min inactivity — acceptable for V1 public launch.
Observability	LangSmith + Sentry	Free tiers. Essential for debugging AI pipelines.

API Endpoint Design

Method	Endpoint	Description	Auth
POST	/api/resume/ingest	Upload master resume → chunk → embed → store in pgvector	Required
DELETE	/api/resume	Delete all user's resume vectors from pgvector	Required
POST	/api/session	Create new scan session with 3–5 job URLs. Returns session_id immediately.	Required
GET	/api/session/{id}	Get session status + job results (traffic lights, scores)	Required
GET	/api/session/{id}/stream	SSE stream of real-time LangGraph node progress	Required
GET	/api/interview/{session_id}	Get interview queue for session	Required
POST	/api/interview/{session_id}/answer	Submit answer → embed → save to pgvector	Required
POST	/api/resume/generate	Generate tailored .docx for a specific job. Returns file stream.	Required
POST	/api/resume/rewrite	Rewrite resume bullets using job description. Returns diff JSON.	Required
POST	/api/resume/rewrite/accept	Accept selected diff changes → generate final .docx	Required
GET	/api/jobs/search	Search jobs via Adzuna API. Params: query, location, remote, salary_min	Required
GET	/api/jobs/{job_id}/score	Score user's current resume against a specific job description	Required
POST	/api/apply/{job_id}	Log application event. Returns: apply_url + triggers resume download.	Required
GET	/api/user/profile	Get user profile (name, email, LinkedIn, application history)	Required
PUT	/api/user/profile	Update user profile fields	Required
GET	/health	Health check endpoint for Render	Public

06 Complete Technology Stack

Every service, cost, and free tier limit in one place.

Layer	Service / Library	Cost	Free Tier Limit	Notes
Frontend	React + Vite	$0	Open source	
Frontend	Vercel	$0	100GB bandwidth/mo	Auto-deploy from GitHub
Backend	FastAPI	$0	Open source	
Backend	Render	$0	Sleeps after 15min	Add /health ping to keep warm if needed
Orchestration	LangGraph	$0	Open source	
AI (PAID)	Gemini 1.5 Pro	Pay-per-use	~$0.007/1K tokens	Your only cost. Rate limit with slowapi.
AI (PAID)	Gemini text-embedding-004	Pay-per-use	~$0.00004/1K chars	Very cheap. Embedded on resume upload only.
Database	Supabase (Postgres)	$0	500MB DB, 2GB storage	Includes pgvector, Auth, Realtime, RLS
Vector Search	pgvector (Supabase)	$0	Part of Supabase free	Replaces Pinecone
Auth	Supabase Auth	$0	50,000 MAU free	Google OAuth + magic links
Realtime	Supabase Realtime	$0	200 concurrent connections	Job status push
Cache/TTL	Supabase pg_cron	$0	Part of Supabase	Replaces Redis
Job APIs	Adzuna API	$0	~250 req/day free	Register at developer.adzuna.com
Job APIs	Greenhouse/Lever JSON	$0	Unlimited (public)	No auth required
Scraping	httpx + Playwright	$0	Open source	Playwright = fallback only

Error Tracking	Sentry	$0	5K errors/mo	sentry.io — add in Day 1
AI Tracing	LangSmith	$0	5K traces/mo	smith.langchain.com
Doc Processing	python-docx	$0	Open source	
Frontend CDN	Cloudflare (optional)	$0	Generous free tier	Add in front of Vercel for extra speed

Cost Projection

Estimated monthly Gemini API cost at 100 active users: Assume 10 resume generations + 3 interview sessions per user/month. ~500K tokens/user/month × 100 users = 50M tokens. At $0.007/1K tokens = ~$350/month. Add rate limiting (5 generations/day/user) to keep this under $50/month at launch. As you grow, Gemini Flash is ~10x cheaper and works well for keyword injection tasks.

07 Sprint Roadmap — 12 Weeks to Public Launch

Organized into 3 phases. Each sprint = 1 week.

Phase 1 — Foundation (Weeks 1–4): Make Core Product Public-Ready

Week	Backend Tasks	Frontend Tasks	Goal
Week 1	Supabase Auth + RLS policies on all tables. JWT middleware in FastAPI. User profile table.	Login page (Supabase Auth UI). Protected routes. Profile page skeleton.	Auth & multi-tenancy solid
Week 2	Replace pgvector query with user_id filter. Test isolation between users. Rate limiting (slowapi).	Resume upload component (react-dropzone). Upload progress indicator. Error states.	Privacy & rate limiting
Week 3	Async job queue (POST returns job_id). Supabase Realtime job status push. Retry logic in LangGraph nodes.	Job URL input form. Real-time traffic light cards (Supabase Realtime subscription). Session history list.	Reliable job scanning
Week 4	Resume generation endpoint. .docx streaming response. Output validation node.	Results page. Resume download button. Feasibility score cards. Interview chat UI.	End-to-end working V1

Phase 2 — New Features (Weeks 5–8): Custom Resume + Job Search + Apply

Week	Backend Tasks	Frontend Tasks	Goal
Week 5	Adzuna API integration. /api/jobs/search endpoint. Filter params (location, remote, salary). Cache results 1h in Supabase.	Jobs search page. Search bar + filters. Job cards with Adzuna data. Save job button.	Free job search live
Week 6	Resume rewrite node in LangGraph. Diff generation (old bullets vs new). /api/resume/rewrite endpoint.	Job description input (manual paste + URL). Rewrite diff viewer (react-diff-viewer). Accept/reject per section.	Custom resume rewrite

Week 7	Resume match score endpoint. Score before/after rewrite. Greenhouse/Lever job fetch endpoints.	Match score visualisation (Recharts donut). Before/after score comparison. Apply button with dual action.	Match scoring + Apply V1
Week 8	User application log table. Profile completion endpoint. Clipboard profile copy API.	Application history in dashboard. Profile completion prompt. One-click apply polish (UX flow).	Full new feature set

Phase 3 — Public Launch Prep (Weeks 9–12): Polish, Performance, Ship

Week	Focus	Key Tasks
Week 9	Observability & Error Handling	Sentry integration (frontend + backend). LangSmith tracing on all LangGraph nodes. Error boundary components in React. User-facing error messages for all failure modes.
Week 10	Performance & Mobile	Responsive CSS for all pages. Lazy loading for job results. Optimistic UI updates. Render cold-start mitigation (/health ping or upgrade plan). pg_cron cleanup job.
Week 11	Security Audit	Review all RLS policies. Input sanitisation on all FastAPI endpoints. File upload size limits. Prompt injection prevention in LLM inputs. Privacy policy page.
Week 12	Public Launch	Custom domain on Vercel. README and contributor docs. Product Hunt / HN launch post. Analytics (Plausible.io — free, privacy-first). Monitor Gemini usage vs budget.

08 Deployment & Go-Live Checklist

Don't ship without these.

Infrastructure

- ■ Supabase project created. RLS enabled on ALL tables.
- ■ pgvector extension enabled in Supabase SQL editor: CREATE EXTENSION vector;
- ■ pg_cron job created to purge scraped_content older than 24 hours
- ■ FastAPI deployed on Render with env vars (Supabase URL/key, Gemini key, LangSmith key)
- ■ React app deployed on Vercel with env vars (Supabase URL/anon key, API base URL)
- ■ Custom domain configured on Vercel

Security

- ■ All FastAPI endpoints require valid Supabase JWT (except /health)
- ■ Rate limiting active: 5 resume generations/day, 10 scans/day per user
- ■ File upload: max 5MB, .docx and .pdf only, virus scan (optional for V1)
- ■ Gemini API key is server-side ONLY — never in frontend code or public repos
- ■ Privacy policy page live with GDPR-compliant data deletion flow

Monitoring

- ■ Sentry DSN configured in both FastAPI (sentry-sdk) and React (@sentry/react)
- ■ LangSmith project created. LANGCHAIN_TRACING_V2=true env var set.
- ■ Gemini API usage alert set in Google Cloud Console (alert at $30, hard limit at $100)
- ■ Supabase dashboard bookmarked for DB size / auth user count monitoring

Testing

- ■ End-to-end test: upload resume → scan 3 jobs → complete interview → generate resume → download
- ■ End-to-end test: search jobs via Adzuna → rewrite resume → accept diff → one-click apply
- ■ Test with 2 separate user accounts: confirm RLS isolates data correctly
- ■ Test Playwright fallback: verify scraper degrades gracefully on blocked URLs
- ■ Test rate limiting: verify 6th generation attempt returns 429 with user-friendly message

Recommended launch sequence: 1) Soft launch to 10 friends → collect feedback → fix critical bugs. 2) Post on Hacker News 'Show HN' — best for this type of technical tool, free, high-quality users. 3) After 100 users and stable usage: upgrade Render to paid tier ($7/mo) to eliminate cold starts. 4) Monitor Gemini costs weekly. If costs exceed $50/mo, switch keyword-injection tasks to Gemini Flash.

This development plan was generated by Claude (Anthropic) · CareerTailor V1.1 · February 2026 · All cost estimates are approximate and subject to change. Verify current pricing on each service's website.